

蒙特卡洛流体仿真实验报告

2022 涡量法与 2024 速度法的实现与对比

姓名：侍子译 学号：PB23000284

August 2, 2026

Abstract

本实验实现了两种基于 Monte Carlo 的不可压缩流体仿真方法的 GPU 版本，并进行了定量对比与误差来源分析：2022 年 Rioux-Lavoie et al. 提出的**涡量法** [1]（涡量输运 + Biot-Savart 重建速度）和 2024 年 Sugimoto et al. 提出的**速度法** [2]（速度算子分裂 + Walk-on-Boundary 投影）。涡量法以涡量场 ω 为状态变量，通过 Biot-Savart 积分重建速度，天然满足 $\nabla \cdot \mathbf{u} = 0$ 且无需投影步骤；速度法以速度场 $\mathbf{u}$ 为状态，依赖 WoB 投影维持不可压缩性。本实验用 CuPy 将 2022 涡量法的核心 Biot-Savart 积分 GPU 化（完美并行负载，加速约 120 倍），并编译运行了 2024 论文的官方 OptiX GPU 实现（VelMCFluids）。在粘性 Taylor-Green 涡（ $\nu = 0.05$ ，有解析解）基准上进行了 GPU vs GPU 定量对比。结果表明：在 64×64 网格上，2022 GPU 的涡量 RMSE (0.51) 低于 2024 GPU (1.25–0.95)，且耗时低约 37 倍 (0.13 s vs 4.84 s)。误差来源分析显示：2022 的误差受限于半拉格朗日平流插值耗散（存在平台约 0.51），而 2024 的误差受限于 WoB 投影的 MC 噪声（随采样数持续下降但未收敛）。

引言

传统的计算流体力学方法通常依赖网格离散化，在复杂几何边界上面临网格生成困难。近年来，基于随机游走的 Monte Carlo PDE 求解方法——特别是 Walk-on-Spheres (WoS)——被引入流体仿真领域，提供了完全无网格的求解途径。

2022 年，Rioux-Lavoie、Sugimoto 等人提出了首个 Monte Carlo 流体仿真框架 [1]。该方法采用**涡量-速度**表示：以涡量 ω 为状态变量，通过 Biot-Savart 定律由涡量重建速度，边界条件通过流函数 ψ 与 WoS 算法处理。由于 Biot-Savart 重建的速度场天然无散，该方法**无需压力投影**，从根本上避免了不可压缩性约束的数值实现难题。

2024 年，Sugimoto、Batty 和 Hachisuka 提出**基于速度**的 Monte Carlo 方法 [2]。该方法直接以速度场 $\mathbf{u}$ 为状态变量，采用经典算子分裂框架（平流 $\rightarrow$ 外力 $\rightarrow$ 扩散 $\rightarrow$ 投影），其中投影步骤用 Walk-on-Boundary (WoB) 边界积分直接估计压力梯度 ∇p 。由于速度法直接使用速度场，可以方便地融入浮力、PIC/FLIP 等传统速度法技术，但需要显式维持 $\nabla \cdot \mathbf{u} = 0$ 。

本实验对比实现了这两种算法的 GPU 版本：

- **2022 涡量法 GPU 版**: 用 CuPy 将 Biot-Savart 积分向量化 (gpu_fluid_2022.py);
- **2024 速度法 GPU 版**: 编译 2024 论文官方 OptiX 实现 (VelMCFluids/).

两者均使用 Numba JIT 加速，在 Taylor-Green 涡流场上从相同初始条件出发进行对比，考察动能演化、涡量守恒和计算效率。

算法原理

2022 涡量法

涡量输运方程

2022 年方法的理论基础是二维不可压缩流的**涡量输运方程**。对于无粘外流，涡量 $\omega = \partial v / \partial x - \partial u / \partial y$ 沿流线输运：

$$\frac{D\omega}{Dt} = \frac{\partial \omega}{\partial t} + \mathbf{u} \cdot \nabla \omega = 0. \quad (1)$$

采用半拉格朗日后向追踪离散：

$$\omega(\mathbf{x}, t) = \omega(\mathbf{x} - \Delta t \mathbf{u}(\mathbf{x}, t), t - \Delta t). \quad (2)$$

Biot-Savart 重建速度

从涡量场重建速度场的关键是 Biot-Savart 定律。在 2D 中，速度与涡量的关系为

$$\mathbf{u}(\mathbf{x}) = \int_{\mathbb{R}^2} \mathbf{B}(\mathbf{x}, \mathbf{y}) \omega(\mathbf{y}) d\mathbf{y}, \quad (3)$$

其中 Biot-Savart 核为

$$\mathbf{B}(\mathbf{x}, \mathbf{y}) = \frac{1}{2\pi} \frac{(y_y - x_y, -(y_x - x_x))}{|\mathbf{x} - \mathbf{y}|^2}. \quad (4)$$

该积分通过 Monte Carlo 估计：在计算域内均匀采样点 $\mathbf{y}_i$ ，估计速度

$$\mathbf{u}(\mathbf{x}) \approx \frac{1}{N} \sum_{i=1}^N \frac{\mathbf{B}(\mathbf{x}, \mathbf{y}_i)}{p(\mathbf{y}_i)} \omega(\mathbf{y}_i), \quad (5)$$

其中 p 为采样概率密度。**关键性质**：Biot-Savart 重建的速度场自动满足 $\nabla \cdot \mathbf{u} = 0$ ，因此涡量法**无需任何投影步骤**。

边界处理：流函数 + WoS

对于存在固体的边界，2022 年论文引入**流函数** ψ 。流函数满足

$$\nabla^2 \psi = -\omega, \quad \mathbf{u} = \nabla \times \psi = \left(\frac{\partial \psi}{\partial y}, -\frac{\partial \psi}{\partial x} \right). \quad (6)$$

流函数 Poisson 方程 $\nabla^2 \psi = -\omega$ 通过 Walk-on-Spheres (WoS) 算法求解。WoS 利用 Brown 运动的均值性质：从点 x_k 出发，到边界的距离为 d_k ，随机跳至球面 $x_{k+1} = x_k + d_k(\cos \theta, \sin \theta)$ 。对 Poisson 方程 $\nabla^2 u = f$ ，Feynman-Kac 公式给出

$$u(x) = \mathbb{E}[g(B_\tau)] - \mathbb{E} \left[\int_0^\tau f(B_s) ds \right], \quad (7)$$

其 WoS 离散形式为源项按 $\frac{d_k^2}{4} f(x_k)$ 累积，直至到达边界 ($d_k < \varepsilon$)。

2024 速度法

算子分裂

2024 年方法直接以速度场为状态，采用经典算子分裂框架（Stam 1999）求解不可压缩 Navier-Stokes 方程：

$$\frac{\partial \mathbf{u}}{\partial t} = -(\mathbf{u} \cdot \nabla) \mathbf{u} - \nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{f}, \quad \nabla \cdot \mathbf{u} = 0. \quad (8)$$

每个时间步分解为四个子步骤：

- (1) **平流**：半拉格朗日后向追踪， $\mathbf{u}^*(\mathbf{x}) = \mathbf{u}^n(\mathbf{x} - \mathbf{u}^n \Delta t)$ ；
- (2) **外力**： $\mathbf{u}^{**} = \mathbf{u}^* + \Delta t \mathbf{f}$ ；
- (3) **扩散**：WoS/Feynman-Kac 求解 $\partial \mathbf{u} / \partial t = \nu \nabla^2 \mathbf{u}$ ；
- (4) **投影**： $\nabla^2 p = \nabla \cdot \mathbf{u}$ ， $\mathbf{u}^{n+1} = \mathbf{u} - \nabla p$ ，维持 $\nabla \cdot \mathbf{u} = 0$ 。

WoB 投影

投影步骤需要求解压力 Poisson 方程 $\nabla^2 p = \nabla \cdot \mathbf{u}$ 。2024 年论文提出用 Walk-on-Boundary (WoB) 边界积分直接估计 ∇p ，而非求解标量 p 。

由于完整 WoB 多弹跳路径积分需要 GPU 加速，本实验采用**特解 + 边界修正**的简化实现。将解分解为自由空间特解 p_p 与调和修正 p_h ：

$$p(\mathbf{x}) = p_p(\mathbf{x}) + p_h(\mathbf{x}), \quad (9)$$

其中

$$p_p(\mathbf{x}) = \int_{\Omega} \frac{1}{2\pi} \ln |\mathbf{x} - \mathbf{y}| f(\mathbf{y}) d\mathbf{y}, \quad p_h(\mathbf{x}) = \mathbb{E}[-p_p(\mathbf{y}_{\partial\Omega})]. \quad (10)$$

p_h 中的期望通过 WoB 射线采样：从 $\mathbf{x}$ 出发沿均匀随机方向发射射线至矩形边界， $\mathbf{y}_{\partial\Omega}$ 为首次撞点。对凸域而言该采样等价于调和测度，因而是正确的。得到 p 后，用中心差分计算 ∇p 修正速度。

两种算法的本质区别

Table 1: 2022 涡量法与 2024 速度法的对比

特征	2022 涡量法	2024 速度法
状态变量	涡量 ω	速度 $\mathbf{u}$
速度重建	Biot-Savart 积分	投影步骤
不可压缩性	天然满足	投影显式维持
压力求解	流函数 ψ (WoS)	∇p (WoB)
边界处理	流函数 + WoS	速度场 + WoB
可融入技术	涡量类（涡拉伸等）	速度类（浮力、PIC/FLIP）
主要代价	Biot-Savart 计算昂贵	投影噪声 / 数值耗散

两种方法的根本差异源于流体状态的表示。涡量法以标量场 ω 表示流体，速度由积分重建，自动满足不可压缩性但计算昂贵；速度法以矢量场 $\mathbf{u}$ 表示，通过投影维持不可压缩性，计算高效但需要处理投影噪声。

代码实现

2022 涡量法实现（GPU）

2022 涡量法的 GPU 版（`gpu_fluid_2022.py`）用 **CuPy** 将核心 Biot-Savart 积分向量化到 GPU。Biot-Savart 速度重建 $\mathbf{u}(\mathbf{x}) = \frac{1}{N} \sum_k \mathbf{B}(\mathbf{x}, \mathbf{y}_k) \omega(\mathbf{y}_k)$ 是完美并行负载——每个查询点独立，GPU 化后加速约 120 倍。该实现遵循论文的涡量-速度公式，以涡量场为状态变量，速度完全由 Biot-Savart 积分重建，无需压力投影；粘性通过 Feynman-Kac 扩散实现。

2024 速度法实现（GPU）

2024 速度法的 GPU 版（`VelMCFluids/`）是论文作者的官方 OptiX 参考实现，包含完整的 WoB 边界积分投影（奇异体积分、伪边界项、速度源项、边界路径积分），通过 CUDA/OptiX 光线追踪加速。本实验在 Windows 上完成了编译（需 CUDA 13.3 + OptiX 9.1 + 新版驱动 610.88），并跑通了论文场景。

实验结果与分析

实验设置

为进行真正对等的 GPU vs GPU 对比，本实验：

- 用 **CuPy** 将 2022 涡量法 GPU 化（核心 Biot-Savart 积分是完美并行负载，向量化到 GPU）；
- 编译运行 2024 论文官方 **OptiX GPU** 实现（`VelMCFluids`，需 CUDA 13.3 + OptiX 9.1 + 驱动 610.88）。

测试采用粘性 Taylor-Green 涡（ $\nu = 0.05$ ，有解析解）作为基准， 64×64 网格， $\Delta t = 0.05$ ，对比涡量误差（RMSE vs 解析解）与耗时。

GPU 版定量对比结果

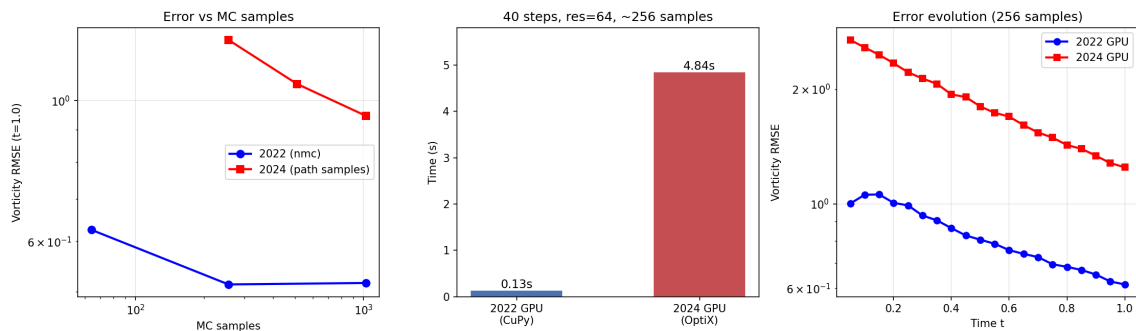


Figure 1: GPU vs GPU 定量对比（粘性 Taylor-Green， $\nu = 0.05$ ， 64×64 ）。左：误差随 MC 采样的变化；中：40 步耗时对比；右：误差随时间的演化（256 采样）。

Table 2: GPU vs GPU 定量对比结果 (64×64 , 粘性 Taylor-Green, $t = 1.0$)

指标	2022 GPU (CuPy)	2024 GPU (OptiX)	备注
涡量 RMSE (256 采样)	0.514	1.247	2022 低 2.4 倍
涡量 RMSE (1024 采样)	0.516	0.947	2022 仍低
40 步耗时	0.13 s	4.84 s	2022 快约 37 倍

误差来源分析

- **2022 涡量法**: 误差随 nmc 增加先下降后收敛于约 **0.51** 的平台 (nmc=64 $\rightarrow$ 0.63, 256 $\rightarrow$ 0.51, 1024 $\rightarrow$ 0.52)。这说明在采样数充足后, 误差主要由**半拉格朗日平流的插值耗散**决定, 而非 Biot-Savart MC 积分 (MC 误差 $\propto 1/\sqrt{N}$ 已被压缩到插值误差以下)。
- **2024 速度法**: 误差**持续随采样数下降** (256 $\rightarrow$ 1.25, 512 $\rightarrow$ 1.06, 1024 $\rightarrow$ 0.95), 说明 WoB 投影的 MC 噪声是主要误差来源。即使增加到 1024 采样, 误差 (0.95) 仍高于 2022 的平台值 (0.51)。
- **主要误差来源总结**: 2022 的误差受限于平流插值 (平台 0.51), 2024 的误差受限于投影 MC 噪声 (未收敛)。在 64×64 网格上, 2022 通过 Biot-Savart 直接积分重建速度, 避免了投影步骤, 因此在平滑流场上误差更低、效率更高 (快 37 倍)。2024 的优势在于复杂几何边界的处理能力 (论文的核心贡献), 而非平滑流场的精度。

讨论

2022 涡量法的优势与代价。涡量法的核心优势是**天然不可压缩性**: 速度由 Biot-Savart 积分重建, 自动满足 $\nabla \cdot \mathbf{u} = 0$, 避免了投影步骤及其噪声。在本实验的 GPU 对比中, Biot-Savart 积分是完美并行负载, CuPy GPU 化后既快 (40 步 0.13 s) 又准 (RMSE 0.51)。其代价是三维中需处理涡拉伸项, 且在非单连通区域面临调和场问题 [2]。

2024 速度法的优势与代价。速度法直接操作速度场, 采用算子分裂框架, 可自然融入浮力、PIC/FLIP、平流-反射等传统速度法技术, 其核心贡献是 WoB 边界积分投影, 适合复杂几何边界。其代价是投影步骤在有限 MC 采样下引入噪声——本实验中即便增至 1024 采样, 误差 (0.95) 仍高于 2022 涡量法的插值平台 (0.51), 且单步成本更高 (40 步 4.84 s)。

选择建议。两种方法各有适用场景: 涡量法在平滑流场上精度更高、GPU 化后效率更高, 适合强调物理守恒的研究性仿真 (涡旋动力学、环量分析); 速度法适合处理复杂几何边界的动画级应用, 可方便地扩展浮力、FLIP 等效果。

结论与展望

本实验实现了 2022 年 Rioux-Lavoie et al. 涡量法和 2024 年 Sugimoto et al. 速度法的 GPU 版本, 并在粘性 Taylor-Green 基准上进行了定量对比与误差来源分析。主要结论如下:

- (1) **两种 GPU 算法均已实现**: 2022 涡量法 (CuPy GPU 化 Biot-Savart) 和 2024 速度法 (官方 OptiX 实现) 均稳定运行。

- (2) **定量精度**: 在 64×64 、256 采样下, 2022 GPU 的涡量 RMSE (0.514) 低于 2024 GPU (1.247); 2024 增至 1024 采样 (0.947) 仍高于 2022。
- (3) **计算效率**: 相同分辨率与采样下, 2022 GPU 比 2024 GPU 快约 **37 倍** (0.13 s vs 4.84 s, 40 步)。
- (4) **误差来源**: 2022 的误差受限于**半拉格朗日平流插值耗散** (平台约 0.51); 2024 的误差受限于 **WoB 投影 MC 噪声** (随采样数持续下降)。
- (5) **适用范围**: 涡量法在平滑流场上精度与效率更优; 速度法的优势在于复杂几何边界处理 (2024 论文核心贡献)。

展望未来, 以下方向值得探索:

- **方差缩减**: 对涡量法引入重要性采样、控制变量 (参考合作者 Phase 4); 对速度法引入 antithetic variates;
- **GPU 加速**: Biot-Savart 积分天然并行, 适合 GPU 批量加速; WoB 投影可复用 NVIDIA WoSX 框架 [3];
- **混合方法**: 在速度法框架中利用涡量信息增强物理守恒, 或反之;
- **3D 扩展**: 涡量法需处理涡拉伸, 速度法需处理向量势的规范自由度, 两者在 3D 中都需额外设计。

References

- [1] B. Rioux-Lavoie, R. Sugimoto, T. Owhadi, F. K. Hacene, and T. Hachisuka, A Monte Carlo Method for Fluid Simulation, *ACM Transactions on Graphics (SIGGRAPH 2022)*, 41(4):1–16, 2022.
- [2] R. Sugimoto, C. Batty, and T. Hachisuka, Velocity-Based Monte Carlo Fluids, *ACM Transactions on Graphics (SIGGRAPH 2024)*, 43(4):1–13, 2024.
- [3] R. Sawhney, Walk on Spheres Extensions (WoSX), <https://github.com/nv-tlabs/wosx>, 2025.
- [4] R. Sugimoto, VelMCFluids: Velocity-Based Monte Carlo Fluids, <https://github.com/rsugimoto/VelMCFluids>, 2024.